AUTOMATED CONTENT PUBLISHING ENGINE

Technical blueprint for Google AI Studio

Target stack

Google AI Studio • GitHub repository • GitHub Actions • Python • Playwright • Gemini API • Blogger API

Goal

Build a low-cost, maintainable workflow that researches useful topics, drafts original articles, captures genuine screenshots, checks quality, and publishes to Blogger with safe controls.

Prepared as a build specification and master prompt. The generated code must be tested in a real GitHub repository and with a test Blogger blog before production use.

Version 1.0 • October 2026

1. Executive brief

Build a Python-first automation repository. Google AI Studio is the coding assistant used to generate and refine the repository; GitHub is the source of truth and GitHub Actions is the scheduled execution environment. The production workflow must run without requiring the user's laptop or a permanently running server.

Primary niche: AI tools, practical digital skills, freelancing, and online income education, with a Zambia-aware perspective where relevant. The goal is useful, original articles—not mass-producing thin pages to manipulate search rankings.

Default safety mode: generate Blogger drafts first. Automatic publication should be an explicit configuration switch and remain disabled until test runs, quality checks, and credentials have been verified.

Success criteria

• A manually triggered GitHub Actions run completes from a clean checkout.

• A scheduled workflow runs at a conservative frequency and avoids duplicate topics.

• Research records source URLs and separates sourced facts from generated suggestions.

• The browser tool captures real screenshots from a permitted, public page; it does not fabricate UI states or bypass access controls.

• Quality checks block publishing when critical requirements fail.

• Blogger integration can create a draft and save the returned post URL/ID in a run report.

• Secrets are stored in GitHub Actions Secrets and never committed to source control.

• The project documents setup, costs, failure handling, and manual recovery.

2. Architecture and data flow

The workflow should be a simple, observable pipeline with explicit intermediate artifacts. Each stage should return structured data and a clear status instead of silently continuing after an error.

Stage	Input	Output / rule
Scheduler	workflow_dispatch; cron	Starts one bounded run; supports a dry-run mode.
Topic selection	Topic queue + RSS feeds	Select one eligible topic; deduplicate using normalized title/keyword.
Research	Topic + approved source list	Research brief with source URLs, dates, quotes kept short, and factual notes.
Article generation	Research brief + editorial rules	HTML article, title, meta description, labels, FAQ, sources, affiliate disclosure if needed.
Screenshot capture	Approved tutorial URLs / steps	Real screenshots, captions, alt text, and attribution/licence notes.
QA gate	Draft + research + assets	Pass/fail report; fail closed on missing sources or unsafe content.
Blogger adapter	Approved HTML + metadata	Draft by default; publish only if explicit config and all gates pass.
Run report	Stage outcomes	JSON summary and artifacts; no tokens or private data in logs.

Recommended first release: one article per run, one run per day at most, manual approval via Blogger draft review, and a manual workflow trigger for testing.

3. Required repository structure

Generate these files as a cohesive project. If a file is not implemented in the first iteration, include a clear stub and document it rather than pretending it works.

```
blogger-content-engine/
■■■ .github/
■   ■■■ workflows/
■       ■■■ content-pipeline.yml
■       ■■■ tests.yml
■■■ config/
■   ■■■ settings.example.yaml
■   ■■■ editorial_policy.md
■■■ data/
■   ■■■ topics.csv
■   ■■■ processed_topics.json
■■■ src/
■   ■■■ main.py
■   ■■■ config.py
■   ■■■ topic_discovery.py
■   ■■■ research.py
■   ■■■ article_writer.py
■   ■■■ screenshot_capture.py
■   ■■■ seo_metadata.py
■   ■■■ quality_checks.py
■   ■■■ blogger_client.py
■   ■■■ state_store.py
■   ■■■ report.py
■■■ templates/
■   ■■■ article_prompt.md
■■■ tests/
■   ■■■ test_quality_checks.py
■   ■■■ test_deduplication.py
■   ■■■ test_blogger_payload.py
■■■ output/.gitkeep
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
■■■ LICENSE
```

File responsibilities

File/module	Responsibility
main.py	Orchestrate stages; stop safely on failure; support --dry-run and --stage options.
topic_discovery.py	Read queue and optional RSS feeds; never scrape Google Search results directly.
research.py	Fetch only allowed sources, record source metadata, and build a research brief.
article_writer.py	Call Gemini through server-side API code; request valid Blogger-compatible HTML and structured output.
screenshot_capture.py	Use Playwright Chromium; timeout limits, consent-aware behavior, no login circumvention.
quality_checks.py	Validate minimum usefulness, links, citations, metadata, HTML, duplicates, disclosures, and forbidden claims.
blogger_client.py	OAuth2 refresh-token flow and Blogger API post creation; draft mode default.
state_store.py	Deduplication and run state; use repository data only for small initial state, with concurrency controls.
report.py	Produce safe JSON/Markdown summary and uploadable artifacts without secrets.

4. Technology choices and integration contracts

Need	Recommended implementation	Important boundary
Execution	GitHub Actions, ubuntu-latest, Python 3.11+	Cron schedules can be delayed; workflow_dispatch must work.
Browser	Playwright Python + Chromium	Install browser dependencies in workflow; cap navigation and capture time.
AI	Gemini API through official SDK or REST	API quota and free-tier terms can change; handle 429/5xx with bounded retries.
Research	RSS feeds, official docs, reputable public sources; optional search provider	Do not assume a free search API exists indefinitely; never scrape Google Search result pages.
Publishing	Blogger API v3	Requires OAuth2 authorization for the blog; create drafts by default.
Assets	Screenshots from actual pages; optional generated/stock images in later phase	Track source and licence; do not copy other publishers' images without permission.
State	CSV/JSON for MVP; optional database later	GitHub workflow filesystems are ephemeral; commit state only through controlled automation or use external storage.
Logs	GitHub Actions summary + artifacts	Redact secrets and avoid logging full API responses containing private data.

Environment variables / GitHub Secrets

Secret / variable	Purpose
GEMINI_API_KEY	Server-side Gemini API key for article generation and optional structured analysis.
BLOGGER_BLOG_ID	Numeric/string blog ID for the target Blogger blog; treat as configuration, not a password.
BLOGGER_CLIENT_ID	OAuth client ID from Google Cloud credentials.
BLOGGER_CLIENT_SECRET	OAuth client secret; store only in GitHub Secrets.
BLOGGER_REFRESH_TOKEN	OAuth refresh token authorized for the target Blogger account/blog.
PUBLISH_MODE	draft or publish; default must be draft.
MAX_ARTICLES_PER_RUN	Default 1; enforce a hard upper limit.
DRY_RUN	When true, create no Blogger post and do not mutate persistent state.

Never commit `.env`, OAuth tokens, API keys, generated private credentials, or local browser profiles. Add them to `.gitignore`. Never put API keys in front-end code, prompt text sent to readers, article HTML, or artifacts.

5. Editorial, SEO, and content-quality specification

The system should optimize for reader usefulness, not a target number of posts. Each article must contribute a concrete benefit: a tested walkthrough, a comparison based on sourced facts, a Zambia-relevant workflow, an original checklist, or a clearly explained example.

Required article fields

• Working title and primary search intent.

• A concise meta description; no keyword stuffing or unsupported promises.

• HTML body with clear H2/H3 headings, short paragraphs, lists, and descriptive links.

• A useful opening that directly addresses the reader's question.

• A practical worked example or step-by-step procedure where appropriate.

• At least two credible sources for factual tutorials/comparisons when available; official product documentation should be preferred for product features and pricing.

• A "last checked" date for volatile features, pricing, eligibility, or platform instructions.

• Image alt text and captions; screenshots must reflect the actual page and steps described.

• Affiliate disclosure near relevant affiliate links, and no affiliate link unless it is explicitly approved in configuration.

• A clear statement when a step, claim, or local eligibility detail could not be independently verified.

Quality gate: hard failures

Check	Block publication if…
Sources	No source list for claims that require verification, sources are inaccessible, or claims contradict the cited source.
Originality/usefulness	Article is mostly paraphrased source material, generic filler, duplicated, or lacks a reader-serving contribution.
Links	Required links are malformed; critical links return errors; affiliate URLs are unapproved.
Metadata/HTML	Title or description missing, HTML malformed, or content is empty/duplicated.
Screenshots	A tutorial depends on screenshots but capture failed, screenshots are blank, or images misrepresent the UI.
Policy/safety	Content contains fabricated results, fake testimonials, misleading earning guarantees, privacy violations, or prohibited claims.
Credentials	Secrets/configuration are missing or a token is exposed in logs or output.
Duplicate guard	Topic fingerprint or near-duplicate check indicates an already processed article.

Google Search policy matters: AI assistance itself is not the issue; generating large volumes of unoriginal pages without added value can be scaled content abuse. Do not automate scraping or republication of other sites. Use sources to verify and synthesize, not to clone.

6. Screenshot and image workflow

Use Playwright to open the actual public page in a headless Chromium browser, wait for the relevant content, and capture a viewport or full-page screenshot. A screenshot should be evidence of the actual tool/site at the time of capture—not a mock-up presented as real.

Minimum requirements

• Set navigation timeout, total stage timeout, and bounded retries.

• Use a predictable viewport and device scale factor so screenshots are consistent.

• Wait for document readiness and a specific selector when one is known; avoid arbitrary long sleeps.

• Check for blank images, browser error pages, consent walls, and login prompts.

• Do not bypass authentication, paywalls, CAPTCHAs, access controls, or rate limits.

• Save screenshot files in a per-run output folder and include alt text, caption, page URL, capture timestamp, and a content hash in the report.

• Use generated images only as illustrations, clearly distinguished from real screenshots. Never claim a generated image is a real app screenshot.

• For screenshots of third-party sites, respect the site's terms and avoid including private account data or personal information.

For the first version, screenshots should be optional unless the article is explicitly a software walkthrough. If a required screenshot fails, the article should fail QA rather than silently inserting a fake or unrelated image.

7. Blogger publishing and authentication

Use Blogger API v3 to create posts. The official `posts.insert` endpoint accepts a `blogId` and supports an `isDraft` parameter. Implement draft creation first and confirm the response includes the post ID and URL before recording success.

```
POST https://www.googleapis.com/blogger/v3/blogs/{blogId}/posts?isDraft=true

Authorization: Bearer <OAuth2 access token>
Content-Type: application/json

{
  "kind": "blogger#post",
  "title": "Article title",
  "content": "<h2>...</h2><p>...</p>",
  "labels": ["AI Tools", "Digital Skills"]
}
```

OAuth setup requirements

• Create/select a Google Cloud project and enable Blogger API v3.

• Configure the OAuth consent screen and OAuth client for the account that owns or manages the blog.

• Obtain a refresh token with the Blogger scope after explicit account authorization.

• Store client ID, client secret, and refresh token as GitHub Actions Secrets.

• Use the refresh token to request short-lived access tokens at runtime; never ask the model to invent credentials.

• Test against a non-production/test blog or create drafts in the target blog before enabling publish mode.

Publishing controls

• Default `PUBLISH_MODE=draft`.

• Require all hard QA checks to pass before calling Blogger API.

• Use a stable idempotency key based on the topic/article fingerprint to prevent duplicate posts on workflow retries.

• If Blogger succeeds but state recording fails, the next run must search/reconcile existing posts before retrying insert.

• Never log OAuth access/refresh tokens or full request headers.

• Automatic publish mode must require both a repository configuration setting and an explicit workflow input/approval mechanism.

8. GitHub Actions workflow requirements

Create a workflow named `content-pipeline.yml` with `workflow_dispatch` and a daily `schedule` using UTC. Add concurrency controls to prevent overlapping runs. Use least-privilege permissions, pin third-party actions to reviewed versions or commit SHAs, and upload run artifacts only when they contain no secrets.

```
name: Blogger Content Pipeline

on:
  workflow_dispatch:
    inputs:
      dry_run:
        description: "Research and draft locally without publishing"
        type: boolean
        default: true
  schedule:
    - cron: "23 5 * * *"  # UTC; adjust after testing

permissions:
```

```
  contents: read

concurrency:
  group: blogger-content-pipeline
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 25
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
          cache: "pip"
      - run: pip install -r requirements.txt
      - run: playwright install --with-deps chromium
      - name: Run pipeline
        env:
          GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }}
          BLOGGER_BLOG_ID: ${{ secrets.BLOGGER_BLOG_ID }}
          BLOGGER_CLIENT_ID: ${{ secrets.BLOGGER_CLIENT_ID }}
          BLOGGER_CLIENT_SECRET: ${{ secrets.BLOGGER_CLIENT_SECRET }}
          BLOGGER_REFRESH_TOKEN: ${{ secrets.BLOGGER_REFRESH_TOKEN }}
          PUBLISH_MODE: draft
          DRY_RUN: ${{ inputs.dry_run }}
        run: python -m src.main
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: content-pipeline-report
          path: output/
          if-no-files-found: warn
          retention-days: 7
```

This is a specification example, not a production-ready workflow by itself. The coding agent must validate YAML syntax, action versions, trigger semantics, permissions, and the selected Python dependency versions before finalizing the repository.

Cost and quota guardrails

• Start with one article per run and a daily schedule.

• Set hard caps on Gemini requests, browser pages, article length, retries, and total runtime.

• Do not assume that a free-tier API has unlimited usage; surface quota errors and stop safely.

• For a public repository, do not expose secrets to untrusted pull-request workflows. A private repository is preferable when the workflow contains publishing credentials.

• Check current GitHub Actions quotas and Gemini API terms before moving from testing to routine production.

9. State, retries, testing, and recovery

GitHub-hosted runners are ephemeral. Files created during a run disappear unless uploaded as artifacts or deliberately saved elsewhere. For the MVP, keep the topic queue in the repository and store processed-topic history in a small JSON file; update it through a controlled commit step or move state to a suitable external store later. Never let two runs write the same state concurrently.

Required test cases

Test	Expected behavior
Missing Gemini key	Stop before API call; report actionable setup error; no post created.
Gemini quota / transient failure	Bounded retries with backoff; then fail safely.
No eligible topic	Successful no-op with a clear report.
Duplicate topic	Skip generation/publishing.
Broken required source	Fail QA or require a replacement source.
Playwright timeout	Retry within limits; block screenshot-dependent article if still failing.
Blogger unauthorized	Stop; never expose tokens; report credential renewal steps.
Blogger creates draft then state write fails	Reconcile using Blogger post lookup/search before attempting another insert.
Dry run enabled	No Blogger API insert and no production state mutation.
Publish mode enabled but QA fails	No publish call; report failed checks.
Workflow cancelled / repeated	Idempotency prevents duplicate article creation.

Operational report

Every run should produce `output/report.json` and a short GitHub Actions job summary containing: run ID, selected topic, stage statuses, source count, screenshot count, QA checks, draft/published URL if present, duration, retry counts, and next action. Redact secrets and avoid dumping entire AI prompts or raw API responses unnecessarily.

10. Phased build plan

Phase	Build	Acceptance test
A — Skeleton	Repository, config validation, topic CSV, CLI, dry-run, tests.	Workflow runs end-to-end without any credentials.
B — Research + writing	RSS/topic intake, research brief, Gemini article generation, source tracking.	Creates an original local HTML draft and structured metadata.
C — Screenshots + QA	Playwright capture, asset manifest, HTML/link/duplicate checks, report.	A tutorial run captures real screenshots and fails correctly when capture is broken.
D — Blogger drafts	OAuth refresh, Blogger client, draft insertion, post URL logging.	A test article appears as a Blogger draft.
E — Scheduling	Cron, concurrency, artifacts, alerts and quotas.	A scheduled run works without manual intervention.
F — Controlled publishing	Explicit publish flag, approvals, idempotency and reconciliation.	Only a passing article can be published; retry cannot duplicate it.

Do not attempt to build every feature in one prompt-response cycle. Have Google AI Studio create the skeleton, then implement and test one phase at a time while keeping the repository runnable after each phase.

11. Copy-ready master prompt for Google AI Studio

Open Google AI Studio Build mode. Paste the prompt below. If it starts creating a generic React dashboard, stop it and reiterate that the deliverable is a Python repository for GitHub Actions, not a hosted web application.

```
You are a senior Python automation engineer and DevOps engineer. Build a complete, testable GitHub repository
named blogger-content-engine. This is NOT primarily a web app or dashboard. The primary deliverable is a
Python CLI pipeline that runs on GitHub Actions and publishes useful, original articles to Blogger.

MISSION
Create a low-cost automated editorial workflow for a blog about AI tools, practical digital skills,
freelancing, and online income education, with Zambia-aware guidance where relevant. The system must
discover/select topics, research reliable sources, generate original articles using the Gemini API, capture
genuine browser screenshots for software tutorials using Playwright, run quality checks, and create Blogger
drafts through Blogger API v3.

NON-NEGOTIABLE ARCHITECTURE
- Python 3.11+; GitHub Actions hosted Ubuntu runner.
- Provide .github/workflows/content-pipeline.yml with workflow_dispatch and a conservative daily schedule.
- Provide a separate test workflow.
- Use Playwright Python with Chromium for real screenshots.
- Use the official Gemini API through server-side Python code. Read GEMINI_API_KEY only from environment
variables.
- Use Blogger API v3 with OAuth2 refresh-token flow. Read BLOGGER_BLOG_ID, BLOGGER_CLIENT_ID,
BLOGGER_CLIENT_SECRET, BLOGGER_REFRESH_TOKEN from environment variables / GitHub Secrets.
- Default PUBLISH_MODE=draft. Publishing must never be enabled by default.
- Support DRY_RUN=true; in dry-run mode, never insert a Blogger post or mutate production state.
- Initial maximum is one article per run, with strict request, retry, page, and runtime limits.
- Use CSV/JSON for a simple MVP queue and state. Document that hosted runner files are ephemeral and
implement safe persistence/reconciliation.
- Never hard-code secrets. Include .gitignore rules for .env, tokens, browser profiles, and private outputs.
- Do not create a React dashboard unless explicitly requested later. Prioritize the Python pipeline and
repository.

REPOSITORY FILES
Create: .github/workflows/content-pipeline.yml; .github/workflows/tests.yml; config/settings.example.yaml;
config/editorial_policy.md; data/topics.csv; data/processed_topics.json; src/main.py; src/config.py;
src/topic_discovery.py; src/research.py; src/article_writer.py; src/screenshot_capture.py;
src/seo_metadata.py; src/quality_checks.py; src/blogger_client.py; src/state_store.py; src/report.py;
templates/article_prompt.md; tests/test_quality_checks.py; tests/test_deduplication.py;
tests/test_blogger_payload.py; requirements.txt; README.md; .gitignore; output/.gitkeep.

PIPELINE
1. Validate config and secrets before making network calls.
2. Select one eligible topic from the queue and optional RSS sources. Do not scrape Google Search result
pages. Avoid duplicate topics by normalized title/keyword fingerprint.
3. Research using official documentation and reputable public sources. Record source URL, title, retrieval
time, and which claims each source supports. If source evidence is insufficient, stop or request review
rather than invent facts.
4. Generate a research brief before writing. Then generate a useful, original article in valid
Blogger-compatible HTML, a title, meta description, labels, FAQ when useful, source list, image alt text, and
affiliate disclosure when needed. Avoid keyword stuffing, fabricated tests, fake testimonials, unsupported
income claims, and unverified feature/pricing/eligibility claims.
5. For screenshot-dependent tutorials, use Playwright to capture genuine pages. Set timeouts, fixed viewport,
bounded retries, and checks for blank/error/login/consent pages. Never bypass CAPTCHAs, paywalls,
authentication, or access controls. Record URL, timestamp, caption, alt text, and hash for each screenshot.
Do not invent a screenshot or present generated art as a real UI capture.
6. Run hard quality gates: sources present and credible; HTML and required metadata valid; key links checked;
duplicate detection; no unapproved affiliate links; screenshot requirements met; no fabricated facts or
misleading earning promises; disclosure present when applicable. Fail closed on critical failures.
7. If DRY_RUN=true, save local artifacts only. Otherwise create a Blogger draft using the official API. Use
isDraft=true by default. Record returned post ID and URL. Only publish when PUBLISH_MODE=publish is
explicitly configured and all gates pass.
```

8. Prevent duplicate posts across retries. If Blogger insert succeeds but local state saving fails, reconcile against Blogger before trying another insert.
9. Write output/report.json and a concise GitHub Actions job summary. Upload safe artifacts for seven days. Never log credentials, access tokens, refresh tokens, or secret request headers.

QUALITY AND SEARCH POLICY
The system must help readers rather than produce mass low-value pages. Do not republish or lightly rewrite sources. Use sources to verify facts and add original walkthroughs, examples, comparisons, checklists, and Zambia-specific context only when verified. Use affiliate URLs only from an explicit approved configuration and include clear disclosure. Track volatile facts with a last-checked date. Respect source terms and image licences.

ENGINEERING QUALITY
- Type hints and docstrings for public functions.
- Structured logging with redaction.
- Clear exceptions and bounded exponential backoff for transient errors and rate limits.
- Unit tests with mocked network/API calls; tests must not require real secrets.
- Validate YAML, Python syntax, imports, and tests before declaring completion.
- Pin or constrain dependencies; use least-privilege GitHub Actions permissions and concurrency controls.
- Never claim the Blogger API, Gemini, or browser workflow has been tested live unless it actually has.
- Include README instructions for Google Cloud Blogger API setup, OAuth refresh-token setup, GitHub Secrets, manual dry run, draft test, scheduled run, cost/quota guardrails, troubleshooting, and safe rollback.
- Provide a .env.example only with placeholder values, never real credentials.

IMPLEMENT IN PHASES
Phase 1: runnable skeleton, config, topic queue, dry-run, tests, workflow.
Phase 2: research + Gemini writing and structured artifacts.
Phase 3: Playwright screenshots + quality checks.
Phase 4: Blogger OAuth + draft creation.
Phase 5: schedule, artifacts, state reconciliation, and optional controlled publishing.

DELIVERY RULE
First create the repository files and a concise implementation plan. Then inspect the entire project for missing imports, inconsistent function contracts, unsafe defaults, YAML errors, and incomplete stubs. Fix these before presenting it. Do not merely describe code—write the actual files. If a feature cannot be implemented or verified in the current environment, state exactly what remains and how to test it. Keep the project runnable after each phase.

12. Setup checklist after AI Studio generates the code

• ■ Review generated files and confirm this is a Python/GitHub Actions repository, not just a web UI.

• ■ Create a GitHub repository. Keep it private during setup because publishing secrets will be configured.

• ■ Use AI Studio's GitHub integration if available for the project, or export the project ZIP and upload/push it to the repository.

• ■ Create a Google Cloud project and enable Blogger API v3.

• ■ Configure OAuth consent and a client; authorize the account that can manage the target Blogger blog; obtain the refresh token using a secure, documented process.

• ■ Add GitHub Actions Secrets: GEMINI_API_KEY, BLOGGER_BLOG_ID, BLOGGER_CLIENT_ID, BLOGGER_CLIENT_SECRET, BLOGGER_REFRESH_TOKEN.

• ■ Run unit tests and a workflow_dispatch dry run before any API publishing call.

• ■ Run a live test that creates one Blogger draft only; inspect HTML, labels, screenshots, links, and formatting manually.

• ■ Check the Actions logs and artifacts for accidental secret exposure.

• ■ Keep PUBLISH_MODE=draft until multiple successful runs are reviewed. Enable automatic publishing only deliberately.

Note on Google AI Studio and GitHub

AI Studio can generate and refine code and supports GitHub sync or ZIP export in its documented Build workflow. That does not mean the content automation itself runs inside AI Studio. After export/sync, GitHub Actions runs the scheduled Python pipeline; API credentials must be configured separately in GitHub Secrets.

13. Official references

• **Google AI Studio — Build apps / GitHub sync and ZIP export**
https://ai.google.dev/gemini-api/docs/aistudio-build-mode

• **Blogger API — Posts: insert (draft support and API request)**
https://developers.google.com/blogger/docs/3.0/reference/posts/insert

• **Blogger API — Using the API and authorization**
https://developers.google.com/blogger/docs/3.0/using

• **GitHub Actions billing and included usage**
https://docs.github.com/en/billing/concepts/product-billing/github-actions

• **Google Search Central — Generative AI content guidance**
https://developers.google.com/search/docs/fundamentals/using-gen-ai-content

• **Google Search Central — Spam policies / scaled content abuse**
https://developers.google.com/search/docs/essentials/spam-policies

• **Playwright for Python — Screenshots**
https://playwright.dev/python/docs/screenshots

• **Gemini API pricing and quotas**
https://ai.google.dev/gemini-api/docs/pricing

Implementation caveat: service quotas, product UI, API behavior, and pricing can change. Verify official documentation at implementation time. The blueprint is a specification, not a claim that a working repository has already been deployed or authorized.